

## Section A Tasks

### Task 1: University Admission System

#### Scenario

You are developing a **university admission system** where student applications must be stored efficiently and retrieved quickly by their **Application ID**.

To achieve this, you decide to use a **Binary Search Tree (BST)**, where each node represents a student's record.

#### Task Requirements

Each student node should contain:

- `ApplicationID` (integer, unique)
- `StudentName` (string)
- `ProgramName` (string)
- `Score` (float, e.g., entry test marks)

#### Core Functionalities to Implement

##### 1. Student Record Management

- `insertStudent(int id, string name, string program, float score)`  
Inserts a new student into the BST based on the `ApplicationID`.
- `searchStudent(int id)`  
Searches and returns a student's details.
- `deleteStudent(int id)`  
Removes the student with the given ID.

##### 2. Tree Inspection

- `displayInorder()` — Displays students sorted by `ApplicationID`.
- `displayPreorder()` — Displays tree structure (root → left → right).
- `displayPostorder()` — Displays tree structure (left → right → root).

### 3. Additional Operations

- `findMin()` — Returns the student with the smallest `ApplicationID`.
- `findMax()` — Returns the student with the largest `ApplicationID`.
- `getHeight()` — Returns the height of the BST.

**Note:** Avoid duplicate `ApplicationIDs` (insertion should ignore or reject duplicates).

```
// Test insertion and search
TEST(StudentBSTTest, InsertAndSearchStudent) {
    StudentBST bst;
    bst.insertStudent(101, "Ali", "CS", 88.5);
    bst.insertStudent(205, "Sara", "AI", 91.0);

    Student s = bst.searchStudent(205);
    EXPECT_EQ(s.StudentName, "Sara");
    EXPECT_EQ(s.ProgramName, "AI");
    EXPECT_FLOAT_EQ(s.Score, 91.0);
}

// Test searching for non-existent student
TEST(StudentBSTTest, SearchNonExistingStudent) {
    StudentBST bst;
    bst.insertStudent(101, "Ali", "CS", 88.5);
    Student s = bst.searchStudent(999);
    EXPECT_EQ(s.StudentName, ""); // empty object if not found
}

// Test deletion of leaf node
TEST(StudentBSTTest, DeleteLeafNode) {
    StudentBST bst;
    bst.insertStudent(50, "A", "CS", 90);
    bst.insertStudent(30, "B", "SE", 85);
```

```

    bst.insertStudent(70, "C", "AI", 95);

    bst.deleteStudent(30);
    EXPECT_EQ(bst.searchStudent(30).StudentName, "");
}

// Test deletion of node with one child
TEST(StudentBSTTest, DeleteNodeWithOneChild) {
    StudentBST bst;
    bst.insertStudent(100, "X", "CS", 80);
    bst.insertStudent(120, "Y", "AI", 85);
    bst.deleteStudent(100);

    EXPECT_EQ(bst.searchStudent(100).StudentName, "");
    EXPECT_EQ(bst.searchStudent(120).StudentName, "Y");
}

// Test inorder traversal order
TEST(StudentBSTTest, InorderTraversalSorted) {
    StudentBST bst;
    bst.insertStudent(50, "A", "CS", 90);
    bst.insertStudent(30, "B", "SE", 80);
    bst.insertStudent(70, "C", "AI", 95);
    vector<int> inorder = bst.getInorderIDs();
    vector<int> expected = {30, 50, 70};
    EXPECT_EQ(inorder, expected);
}

// Test finding minimum and maximum
TEST(StudentBSTTest, FindMinAndMax) {
    StudentBST bst;
    bst.insertStudent(40, "A", "CS", 88);
    bst.insertStudent(20, "B", "SE", 91);
    bst.insertStudent(70, "C", "AI", 77);
    EXPECT_EQ(bst.findMin().ApplicationID, 20);
    EXPECT_EQ(bst.findMax().ApplicationID, 70);
}

// Test tree height
TEST(StudentBSTTest, TreeHeight) {

```

```

StudentBST bst;
bst.insertStudent(50, "A", "CS", 90);
bst.insertStudent(30, "B", "SE", 85);
bst.insertStudent(70, "C", "AI", 95);
bst.insertStudent(20, "D", "DS", 75);
EXPECT_EQ(bst.getHeight(), 3);
}

```

## Task 2: Advanced BST Task: File System Indexing Simulation Using BST

### Scenario Overview

You are part of a development team building a **lightweight file indexing service** for a prototype operating system.

This system keeps track of files using a **Binary Search Tree (BST)** data structure. Each node in the BST represents a **file**, identified by a **unique File ID** (integer).

The file system frequently needs to:

- Locate a file quickly by its ID
- Track when each file was last accessed
- Remove outdated or unused files
- Generate reports such as “Which file was accessed least recently?”

Your task is to design and implement the BST to efficiently support these operations, while ensuring the BST remains correctly structured after updates.

### Data Model

Each node in the BST should store:

FileID (int) → unique key used for BST ordering

FileName (string) → descriptive file name

LastAccessTime (int or datetime) → timestamp of the most recent access

FileSize (int) → size in KB/MB

## Core Functionalities

### Insertion of a New File

- Insert a new file record (FileID, FileName, FileSize, LastAccessTime).
- BST must maintain ordering based on **FileID**.
- Prevent duplicate FileIDs (if duplicate found, update LastAccessTime instead).

### File Search

- Given a **FileID**, locate the corresponding file record.
- Return all details of the file.
- If not found, return an appropriate message.
- Update the file's **LastAccessTime** automatically after each successful search (simulating file access).

### File Deletion

- Delete a file by **FileID**.
- The BST must remain structurally valid after deletion (handle leaf, single-child, and two-child cases).

### Retrieve Least Recently Accessed File

- Implement a function **getLeastRecentlyAccessed()** that traverses the BST to find the file with the **oldest LastAccessTime**.

### Display All Files in Ascending Order of FileID

- Perform an **in-order traversal** to print all file records sorted by FileID.

### Display All Files Sorted by LastAccessTime

- Perform a **custom traversal** (not BST-ordered) to list files by ascending or descending `LastAccessTime`.
- This should demonstrate traversal beyond simple in-order patterns.

## Extract IDs

- You will have to create a function that simply returns an int array containing the File IDs. This is needed for the test cases late

## Test Cases

```
TEST(FileSystemBSTTest, InsertAndSearchBasic) {
    FileSystemBST fs;

    EXPECT_TRUE(fs.insert(105, "report.docx", 256, "10:15"));
    EXPECT_TRUE(fs.insert(110, "notes.txt", 32, "10:20"));
    EXPECT_TRUE(fs.insert(101, "config.sys", 5, "09:30"));

    // Duplicate insert should fail or update access time
    EXPECT_FALSE(fs.insert(105, "report.docx", 256, "10:25"));

    FileNode* found = fs.search(110);
    ASSERT_NE(found, nullptr);
    EXPECT_EQ(found->fileName, "notes.txt");
    EXPECT_EQ(found->fileSize, 32);

    EXPECT_EQ(fs.search(999), nullptr); // nonexistent file
}
```

```
TEST(FileSystemBSTTest, InOrderTraversalSorted) {
    FileSystemBST fs;
    fs.insert(105, "report.docx", 256, "10:15");
    fs.insert(110, "notes.txt", 32, "10:20");
    fs.insert(101, "config.sys", 5, "09:30");

    auto inorder = extractIDs(fs.getInOrder());
    int expected[] = {101, 105, 110};
    EXPECT_EQ(inorder, expected);
}
```

```

TEST(FileSystemBSTTest, DeleteOperations) {
    FileSystemBST fs;
    fs.insert(105, "report.docx", 256, "10:15");
    fs.insert(110, "notes.txt", 32, "10:20");
    fs.insert(101, "config.sys", 5, "09:30");

    // Delete leaf
    EXPECT_TRUE(fs.deleteFile(110));
    // Delete one-child node
    EXPECT_TRUE(fs.deleteFile(101));
    // Delete root (two children)
    EXPECT_TRUE(fs.deleteFile(105));

    EXPECT_EQ(fs.search(105), nullptr);
}

```

```

TEST(FileSystemBSTTest, LeastRecentlyAccessedFile) {
    FileSystemBST fs;
    fs.insert(105, "report.docx", 256, "10:15");
    fs.insert(110, "notes.txt", 32, "10:20");
    fs.insert(101, "config.sys", 5, "09:10");

    FileNode* least = fs.getLeastRecentlyAccessed();
    ASSERT_NE(least, nullptr);
    EXPECT_EQ(least->fileID, 101);
}

```

```

TEST(FileSystemBSTTest, SortByAccessTime) {
    FileSystemBST fs;
    fs.insert(105, "report.docx", 256, "10:15");
    fs.insert(110, "notes.txt", 32, "10:20");
    fs.insert(101, "config.sys", 5, "09:30");

    auto sorted = extractIDs(fs.GetFilesSortedByAccessTime());
    int expected[] = {101, 105, 110};
}

```

```
    ASSERT_EQ(sorted.size(), 3);  
    EXPECT_TRUE(std::equal(sorted.begin(), sorted.end(), expected));  
}
```

```
TEST(FileSystemBSTTest, HeightMeasurement) {  
    FileSystemBST fs;  
    fs.insert(10, "a.txt", 1, "09:00");  
    fs.insert(5, "b.txt", 1, "09:01");  
    fs.insert(15, "c.txt", 1, "09:02");  
    fs.insert(2, "d.txt", 1, "09:03");  
    fs.insert(7, "e.txt", 1, "09:04");  
  
    EXPECT_EQ(fs.getHeight(), 3);  
}
```